02_potential_outcomes

May 4, 2026

1 The Potential Outcomes Framework (Rubin Causal Model)

1.1 Learning Objectives

By the end of this notebook, you will be able to: - Define potential outcomes and the fundamental problem of causal inference - Calculate individual and average treatment effects using the potential outcomes framework - Understand the role of randomization in causal inference - Recognize selection bias and its impact on causal estimates - Implement simulations to demonstrate key concepts

---

1.2 Overview

The **Potential Outcomes Framework**, also known as the **Rubin Causal Model** (after statistician Donald Rubin), provides a formal way to define causal effects. It's one of the two main frameworks for causal inference (the other being DAGs, which we'll cover next).

Key idea: Each unit (person, company, region) has multiple *potential outcomes* - one for each possible treatment they could receive. The causal effect is the comparison between these potential outcomes.

1.2.1 Quick Theory Recap

For a binary treatment (T = 1 for treated, T = 0 for control):

- **Y(1)**: Potential outcome if unit receives treatment
- **Y(0)**: Potential outcome if unit does NOT receive treatment
- **Individual Treatment Effect (ITE)**: = Y(1) - Y(0) for unit i
- **Average Treatment Effect (ATE)**: E[Y(1) - Y(0)]

The Fundamental Problem: We only observe one potential outcome per unit. If a person receives treatment, we see Y(1) but never see Y(0).

Let's explore this with code and simulations!

1.3 Setup

First, let's ensure all required libraries are installed. If you haven't already, install dependencies from the repository's `requirements.txt` file.

[1]:
```python
# Import libraries
import os, sys, yaml
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from typing import Tuple

# Set style and random seed for reproducibility
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)
np.random.seed(42)

print("Libraries imported successfully!")
print(f"NumPy version: {np.__version__}")
print(f"Pandas version: {pd.__version__}")
```

```
Libraries imported successfully!
NumPy version: 2.3.3
Pandas version: 2.3.3
```

[2]:
```python
# Install dependencies (run this cell if you haven't installed requirements.txt)
# Uncomment the line below if running for the first time
# !pip install -r ../requirements.txt

# Alternatively, install only the libraries needed for this notebook:
# !pip install numpy>=1.21.0 pandas>=1.3.0 matplotlib>=3.4.0 seaborn>=0.11.0
```

1.4 Part 1: Potential Outcomes in the Ideal World

Let's start by imagining an **impossible scenario**: we can observe **both** potential outcomes for each person. This will help us understand what causal effects actually are.

1.4.1 Example: Job Training Program

Imagine we have 10 people. We want to know: Does a job training program increase earnings?

In our imaginary world, we can observe: - **Y(1)**: What each person would earn WITH the training - **Y(0)**: What each person would earn WITHOUT the training

[3]:
```python
# Create hypothetical data where we observe BOTH potential outcomes
# (This is impossible in reality - we're doing it for illustration)

n = 10
data_ideal = pd.DataFrame({
    'person_id': range(1, n + 1),
```

```
    'Y0': [35, 38, 32, 40, 36, 34, 37, 33, 39, 35],  # Earnings without␣
↪training (in $1000s)
    'Y1': [40, 44, 35, 46, 41, 38, 42, 36, 45, 39],  # Earnings with training␣
↪(in $1000s)
})

# Calculate individual treatment effects
data_ideal['ITE'] = data_ideal['Y1'] - data_ideal['Y0']

print("Ideal World: We observe BOTH potential outcomes\n")
print(data_ideal)
print("\n" + "="*60)
```

```
Ideal World: We observe BOTH potential outcomes

   person_id  Y0  Y1  ITE
0          1  35  40    5
1          2  38  44    6
2          3  32  35    3
3          4  40  46    6
4          5  36  41    5
5          6  34  38    4
6          7  37  42    5
7          8  33  36    3
8          9  39  45    6
9         10  35  39    4

============================================================
```

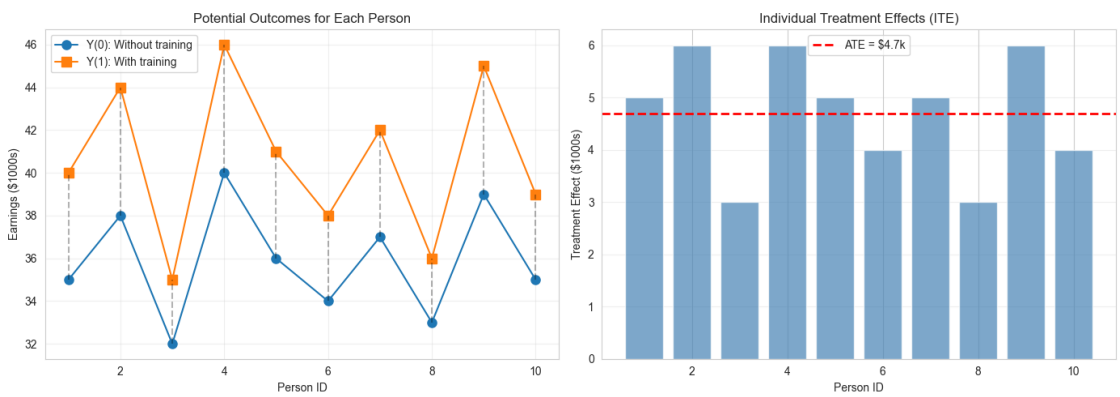
1.4.2 Individual Treatment Effects (ITE)

Notice that the treatment effect **varies by person**: - Person 1: Gains $5k from training - Person 2: Gains $6k from training - Person 3: Gains only $3k

This is **treatment effect heterogeneity** - different people benefit differently from the same treatment.

[4]:
```
# Visualize individual treatment effects
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Left panel: Potential outcomes for each person
x = data_ideal['person_id']
axes[0].plot(x, data_ideal['Y0'], 'o-', label='Y(0): Without training',␣
↪markersize=8)
axes[0].plot(x, data_ideal['Y1'], 's-', label='Y(1): With training',␣
↪markersize=8)
for i in range(len(data_ideal)):
    axes[0].plot([x.iloc[i], x.iloc[i]],
```

```
                  [data_ideal['Y0'].iloc[i], data_ideal['Y1'].iloc[i]],
                  'k--', alpha=0.3)
axes[0].set_xlabel('Person ID')
axes[0].set_ylabel('Earnings ($1000s)')
axes[0].set_title('Potential Outcomes for Each Person')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Right panel: Distribution of treatment effects
axes[1].bar(x, data_ideal['ITE'], color='steelblue', alpha=0.7)
axes[1].axhline(data_ideal['ITE'].mean(), color='red', linestyle='--',
                linewidth=2, label=f'ATE = ${data_ideal["ITE"].mean():.1f}k')
axes[1].set_xlabel('Person ID')
axes[1].set_ylabel('Treatment Effect ($1000s)')
axes[1].set_title('Individual Treatment Effects (ITE)')
axes[1].legend()
axes[1].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()
```



1.4.3 Average Treatment Effect (ATE)

While individual effects vary, we often care about the **average effect** across the population.

[5]:
```
# Calculate Average Treatment Effect
ATE = data_ideal['ITE'].mean()

print(f"Average Treatment Effect (ATE): ${ATE:.2f}k")
print(f"\nInterpretation: On average, the job training program increases")
print(f"earnings by ${ATE:.2f}k (or ${ATE*1000:.0f}).")
print(f"\nRange of ITEs: ${data_ideal['ITE'].min():.0f}k to ${data_ideal['ITE'].
↪max():.0f}k")
```

```
print(f"Standard deviation: ${data_ideal['ITE'].std():.2f}k")
```

```
Average Treatment Effect (ATE): $4.70k

Interpretation: On average, the job training program increases
earnings by $4.70k (or $4700).

Range of ITEs: $3k to $6k
Standard deviation: $1.16k
```

1.5 Part 2: The Fundamental Problem of Causal Inference

Now let's face reality: **We can never observe both potential outcomes for the same person.**

- If Alice gets training, we see Y(1) but not Y(0)
- If Bob doesn't get training, we see Y(0) but not Y(1)

This is called the **Fundamental Problem of Causal Inference**.

[6]:
```
# Simulate a realistic scenario: some people get training, others don't
# We only observe ONE potential outcome per person

# Randomly assign treatment (for now - we'll make it non-random later)
data_real = data_ideal.copy()
data_real['T'] = np.random.binomial(1, 0.5, n)  # 50% get training

# Observed outcome: Y(1) if treated, Y(0) if not
data_real['Y_observed'] = np.where(data_real['T'] == 1,
                                   data_real['Y1'],
                                   data_real['Y0'])

# Create missing data (what we DON'T observe)
data_real['Y0_obs'] = np.where(data_real['T'] == 0, data_real['Y0'], np.nan)
data_real['Y1_obs'] = np.where(data_real['T'] == 1, data_real['Y1'], np.nan)

print("Reality: We only observe ONE potential outcome per person\n")
print(data_real[['person_id', 'T', 'Y0_obs', 'Y1_obs', 'Y_observed', 'ITE']].
 ↪to_string())
print("\n? = Missing (cannot be observed)")
print("\nNote: We show ITE here for illustration, but in reality we CANNOT␣
 ↪calculate it!")
```

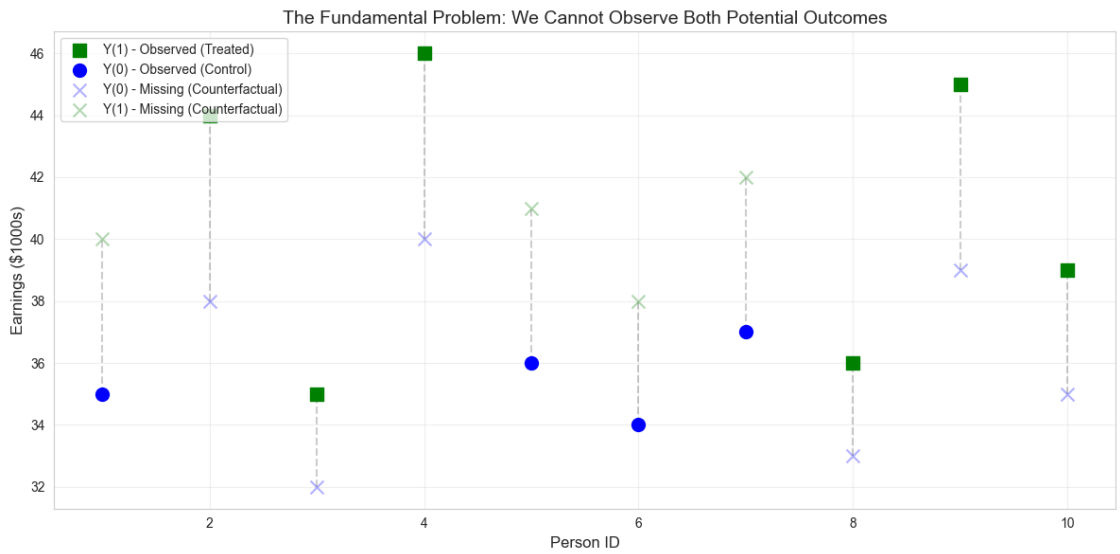
```
Reality: We only observe ONE potential outcome per person

   person_id  T  Y0_obs  Y1_obs  Y_observed  ITE
0          1  0    35.0     NaN          35    5
1          2  1     NaN    44.0          44    6
2          3  1     NaN    35.0          35    3
```

```
3          4  1     NaN    46.0          46    6
4          5  0    36.0     NaN          36    5
5          6  0    34.0     NaN          34    4
6          7  0    37.0     NaN          37    5
7          8  1     NaN    36.0          36    3
8          9  1     NaN    45.0          45    6
9         10  1     NaN    39.0          39    4

? = Missing (cannot be observed)

Note: We show ITE here for illustration, but in reality we CANNOT calculate it!
```

1.5.1 Visualizing the Missing Data Problem

[7]:
```
# Visualize what we observe vs what's missing
fig, ax = plt.subplots(figsize=(12, 6))

x = data_real['person_id']
treated = data_real['T'] == 1
control = data_real['T'] == 0

# Plot observed outcomes
ax.scatter(x[treated], data_real.loc[treated, 'Y1'],
           s=100, marker='s', color='green', label='Y(1) - Observed (Treated)',␣
 ↪zorder=3)
ax.scatter(x[control], data_real.loc[control, 'Y0'],
           s=100, marker='o', color='blue', label='Y(0) - Observed (Control)',␣
 ↪zorder=3)

# Plot missing (counterfactual) outcomes with X
ax.scatter(x[treated], data_real.loc[treated, 'Y0'],
           s=100, marker='x', color='blue', alpha=0.3, label='Y(0) - Missing␣
 ↪(Counterfactual)', zorder=2)
ax.scatter(x[control], data_real.loc[control, 'Y1'],
           s=100, marker='x', color='green', alpha=0.3, label='Y(1) - Missing␣
 ↪(Counterfactual)', zorder=2)

# Connect observed and counterfactual for each person
for i in range(len(data_real)):
    ax.plot([x.iloc[i], x.iloc[i]],
            [data_real['Y0'].iloc[i], data_real['Y1'].iloc[i]],
            'k--', alpha=0.2, zorder=1)

ax.set_xlabel('Person ID', fontsize=12)
ax.set_ylabel('Earnings ($1000s)', fontsize=12)
ax.set_title('The Fundamental Problem: We Cannot Observe Both Potential␣
 ↪Outcomes', fontsize=14)
```

```
ax.legend(loc='upper left')
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\nKey insight: For each person, we see one solid point (observed) and␣
↪one X (counterfactual).")
print("The dashed lines show the individual treatment effects we CANNOT␣
↪calculate.")
```




```
Key insight: For each person, we see one solid point (observed) and one X
(counterfactual).
The dashed lines show the individual treatment effects we CANNOT calculate.
```

1.6 Part 3: Naive Comparison (Why It Fails)

A natural thought: "Can't we just compare the average outcome of treated people to the average outcome of control people?"

Let's try it with **random assignment** first, then see what happens with **non-random assignment**.

1.6.1 Scenario A: Random Assignment (Works!)

[8]:
```
# Calculate naive estimator with random treatment assignment
treated_mean = data_real[data_real['T'] == 1]['Y_observed'].mean()
control_mean = data_real[data_real['T'] == 0]['Y_observed'].mean()
naive_estimate = treated_mean - control_mean

print("SCENARIO A: Random Treatment Assignment\n")
print(f"Average outcome for treated:  ${treated_mean:.2f}k")
print(f"Average outcome for control:  ${control_mean:.2f}k")
print(f"Naive estimate (difference):  ${naive_estimate:.2f}k")
print(f"\nTrue ATE (from ideal world):  ${ATE:.2f}k")
print(f"Estimation error:             ${abs(naive_estimate - ATE):.2f}k")
print(f"\nWith random assignment, the naive estimator is close to the true ATE!
 ↪")
print(f"(Small error is due to random sampling variation)")
```

```
SCENARIO A: Random Treatment Assignment

Average outcome for treated:  $40.83k
Average outcome for control:  $35.50k
Naive estimate (difference):  $5.33k

True ATE (from ideal world):  $4.70k
Estimation error:             $0.63k

With random assignment, the naive estimator is close to the true ATE!
(Small error is due to random sampling variation)
```

1.6.2 Scenario B: Selection Bias (Fails!)

Now let's simulate what happens when treatment assignment is **not random**.

Suppose more motivated people are more likely to enroll in training. Let's say people who would earn more anyway (high Y(0)) are more likely to get training.

[9]:
```
# Create selection bias: people with higher baseline earnings (Y0) more likely␣
 ↪to get training
data_biased = data_ideal.copy()

# Treatment probability increases with baseline earnings
# Logistic function: P(T=1) depends on Y(0)
baseline_earnings_normalized = (data_biased['Y0'] - data_biased['Y0'].mean()) /␣
 ↪data_biased['Y0'].std()
prob_treatment = 1 / (1 + np.exp(-baseline_earnings_normalized))

# Assign treatment based on these probabilities
data_biased['T'] = np.random.binomial(1, prob_treatment)
```

```
# Observed outcome
data_biased['Y_observed'] = np.where(data_biased['T'] == 1,
                                     data_biased['Y1'],
                                     data_biased['Y0'])

print("SCENARIO B: Selection Bias (Non-Random Assignment)\n")
print(data_biased[['person_id', 'Y0', 'Y1', 'T', 'Y_observed']].to_string())
print(f"\nNote: People with higher Y(0) are more likely to have T=1")
```

```
SCENARIO B: Selection Bias (Non-Random Assignment)

   person_id  Y0  Y1  T  Y_observed
0          1  35  40  0          35
1          2  38  44  0          38
2          3  32  35  1          35
3          4  40  46  1          46
4          5  36  41  1          41
5          6  34  38  0          34
6          7  37  42  1          42
7          8  33  36  0          33
8          9  39  45  1          45
9         10  35  39  0          35

Note: People with higher Y(0) are more likely to have T=1
```

[10]:
```
# Calculate naive estimator with selection bias
treated_mean_biased = data_biased[data_biased['T'] == 1]['Y_observed'].mean()
control_mean_biased = data_biased[data_biased['T'] == 0]['Y_observed'].mean()
naive_estimate_biased = treated_mean_biased - control_mean_biased

# Also calculate the baseline difference
baseline_diff = data_biased[data_biased['T'] == 1]['Y0'].mean() -␣
↪data_biased[data_biased['T'] == 0]['Y0'].mean()

print("Results with Selection Bias:\n")
print(f"Average outcome for treated:   ${treated_mean_biased:.2f}k")
print(f"Average outcome for control:   ${control_mean_biased:.2f}k")
print(f"Naive estimate (difference):   ${naive_estimate_biased:.2f}k")
print(f"\nTrue ATE:                      ${ATE:.2f}k")
print(f"BIAS:                          ${naive_estimate_biased - ATE:.2f}k")
print(f"\nBaseline difference E[Y(0)|T=1] - E[Y(0)|T=0]: ${baseline_diff:.2f}k")
print(f"\n  The naive estimator OVERESTIMATES the treatment effect!")
print(f"This is because treated people would have earned more even without␣
↪training.")
```

```
Results with Selection Bias:
```

```
Average outcome for treated:   $41.80k
Average outcome for control:   $35.00k
Naive estimate (difference):   $6.80k

True ATE:                      $4.70k
BIAS:                          $2.10k

Baseline difference E[Y(0)|T=1] - E[Y(0)|T=0]: $1.80k

 The naive estimator OVERESTIMATES the treatment effect!
This is because treated people would have earned more even without training.
```

1.6.3 Decomposing the Bias

The naive estimator can be decomposed as:

E[Y|T=1] - E[Y|T=0] = ATE + Selection Bias

Where: - **ATE** = E[Y(1) - Y(0)] (what we want) - **Selection Bias** = E[Y(0)|T=1] - E[Y(0)|T=0] (difference in baseline outcomes)

Let's verify this:

[11]:
```
# Decompose the bias
# The correct decomposition is:
# Naive Estimate = True ATE + Selection Bias
# Where Selection Bias is the total bias, not just baseline difference

# Total selection bias (includes all sources of bias)
total_selection_bias = naive_estimate_biased - ATE

# Baseline difference E[Y(0)|T=1] - E[Y(0)|T=0]
# This is ONE component of selection bias
baseline_diff_component = baseline_diff

print("Bias Decomposition:\n")
print(f"Naive Estimate                = ${naive_estimate_biased:.2f}k")
print(f"True ATE                      = ${ATE:.2f}k")
print(f"Total Selection Bias          = ${total_selection_bias:.2f}k")
print(f"\nVerification: ATE + Selection Bias = Naive Estimate")
print(f"              {ATE:.2f} + {total_selection_bias:.2f} = {ATE +␣
↪total_selection_bias:.2f}k")
print(f"Naive Estimate:                     {naive_estimate_biased:.2f}k")
print(f"\n  The decomposition holds!")

print(f"\n" + "="*60)
print(f"Note: Baseline difference E[Y(0)|T=1] - E[Y(0)|T=0] =␣
↪${baseline_diff_component:.2f}k")
```

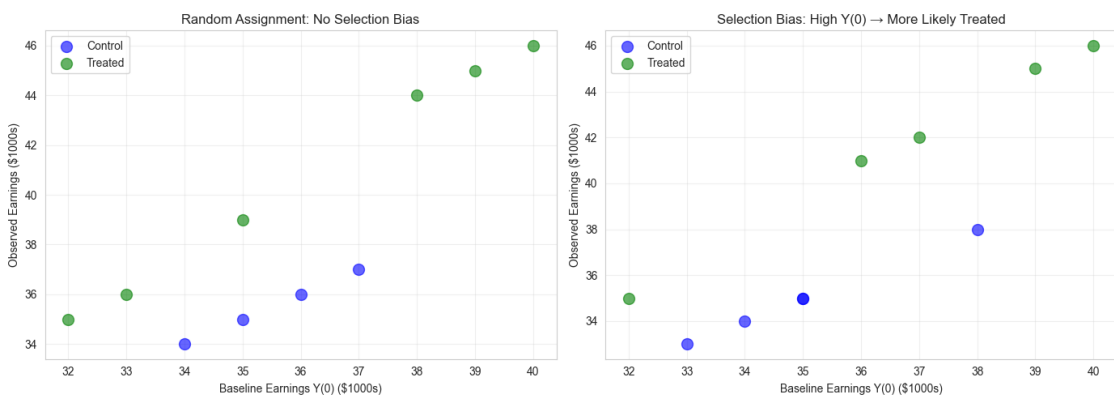
```
print(f"This is close to, but not exactly equal to, the total bias due to:")
print(f"  - Small sample size (n={n})")
print(f"  - Sampling variability")
print(f"  - Heterogeneous treatment effects")
print(f"In large samples, these would converge.")
```

```
Bias Decomposition:

Naive Estimate                = $6.80k
True ATE                      = $4.70k
Total Selection Bias          = $2.10k

Verification: ATE + Selection Bias = Naive Estimate
              4.70 + 2.10 = 6.80k
Naive Estimate:                     6.80k

 The decomposition holds!

============================================================
Note: Baseline difference E[Y(0)|T=1] - E[Y(0)|T=0] = $1.80k
This is close to, but not exactly equal to, the total bias due to:
  - Small sample size (n=10)
  - Sampling variability
  - Heterogeneous treatment effects
In large samples, these would converge.
```

1.6.4 Visualizing Selection Bias

[12]:
```
# Compare random vs biased assignment
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Left: Random assignment
treated_random = data_real['T'] == 1
control_random = data_real['T'] == 0
axes[0].scatter(data_real.loc[control_random, 'Y0'],
                data_real.loc[control_random, 'Y_observed'],
                s=100, alpha=0.6, label='Control', color='blue')
axes[0].scatter(data_real.loc[treated_random, 'Y0'],
                data_real.loc[treated_random, 'Y_observed'],
                s=100, alpha=0.6, label='Treated', color='green')
axes[0].set_xlabel('Baseline Earnings Y(0) ($1000s)')
axes[0].set_ylabel('Observed Earnings ($1000s)')
axes[0].set_title('Random Assignment: No Selection Bias')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Right: Biased assignment
```

```
treated_biased = data_biased['T'] == 1
control_biased = data_biased['T'] == 0
axes[1].scatter(data_biased.loc[control_biased, 'Y0'],
                data_biased.loc[control_biased, 'Y_observed'],
                s=100, alpha=0.6, label='Control', color='blue')
axes[1].scatter(data_biased.loc[treated_biased, 'Y0'],
                data_biased.loc[treated_biased, 'Y_observed'],
                s=100, alpha=0.6, label='Treated', color='green')
axes[1].set_xlabel('Baseline Earnings Y(0) ($1000s)')
axes[1].set_ylabel('Observed Earnings ($1000s)')
axes[1].set_title('Selection Bias: High Y(0) → More Likely Treated')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\nKey difference: With selection bias (right), treated people cluster at␣
↪higher Y(0).")
print("This makes them different from control group at baseline, confounding␣
↪the comparison.")
```




```
Key difference: With selection bias (right), treated people cluster at higher
Y(0).
This makes them different from control group at baseline, confounding the
comparison.
```

1.7 Part 4: The Power of Randomization

Let's see why randomization is the "gold standard" for causal inference by simulating a larger randomized experiment.

[13]:
```python
def simulate_experiment(n: int,
                        treatment_effect: float,
                        random_assignment: bool = True) -> Tuple[float, float]:
    """
    Simulate an experiment with n people.

    Parameters
    ----------
    n : int
        Number of people
    treatment_effect : float
        True average treatment effect
    random_assignment : bool
        If True, randomly assign treatment. If False, create selection bias.

    Returns
    -------
    estimated_effect : float
        Estimated treatment effect from the experiment
    true_effect : float
        True average treatment effect
    """
    # Generate baseline outcomes Y(0) from normal distribution
    Y0 = np.random.normal(35, 5, n)

    # Generate treatment effects (heterogeneous, but average is␣
↪treatment_effect)
    individual_effects = np.random.normal(treatment_effect, 1, n)
    Y1 = Y0 + individual_effects

    # Assign treatment
    if random_assignment:
        T = np.random.binomial(1, 0.5, n)
    else:
        # Selection bias: higher Y(0) more likely to be treated
        Y0_normalized = (Y0 - Y0.mean()) / Y0.std()
        prob_T = 1 / (1 + np.exp(-Y0_normalized))
        T = np.random.binomial(1, prob_T)

    # Observe outcomes
    Y_obs = np.where(T == 1, Y1, Y0)

    # Estimate treatment effect
    estimated_effect = Y_obs[T == 1].mean() - Y_obs[T == 0].mean()
    true_effect = individual_effects.mean()

    return estimated_effect, true_effect
```

```
print("Simulation function defined!")
```

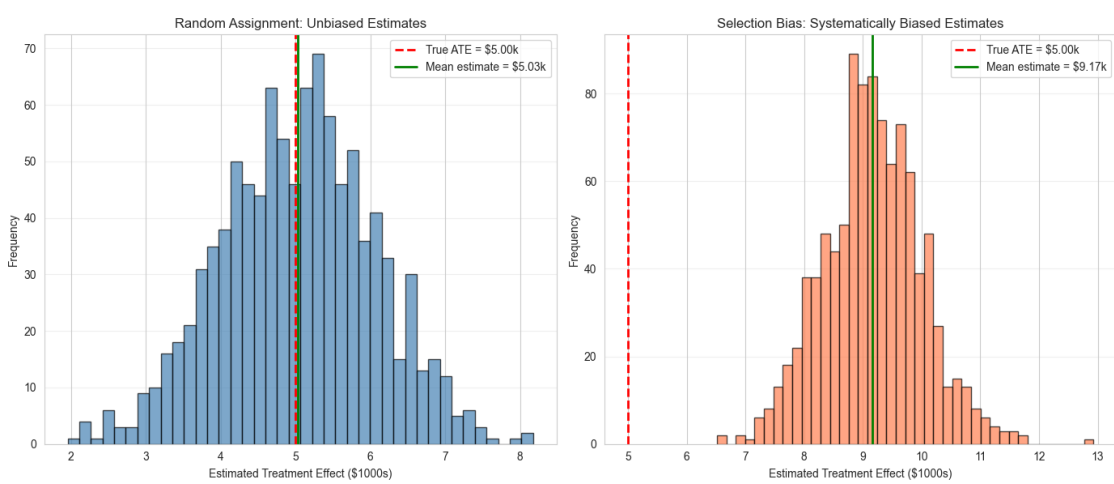
```
Simulation function defined!
```

[14]:
```
# Run many experiments to see the distribution of estimates
n_experiments = 1000
n_people = 100
true_ate = 5.0

# Random assignment experiments
estimates_random = []
for _ in range(n_experiments):
    est, _ = simulate_experiment(n_people, true_ate, random_assignment=True)
    estimates_random.append(est)

# Biased assignment experiments
estimates_biased = []
for _ in range(n_experiments):
    est, _ = simulate_experiment(n_people, true_ate, random_assignment=False)
    estimates_biased.append(est)

estimates_random = np.array(estimates_random)
estimates_biased = np.array(estimates_biased)

print(f"Ran {n_experiments} simulated experiments with {n_people} people each.")
print(f"\nTrue ATE: ${true_ate:.2f}k")
```

```
Ran 1000 simulated experiments with 100 people each.

True ATE: $5.00k
```

[15]:
```
# Visualize the distribution of estimates
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Left: Random assignment
axes[0].hist(estimates_random, bins=40, alpha=0.7, color='steelblue',␣
 ↪edgecolor='black')
axes[0].axvline(true_ate, color='red', linestyle='--', linewidth=2,␣
 ↪label=f'True ATE = ${true_ate:.2f}k')
axes[0].axvline(estimates_random.mean(), color='green', linestyle='-',␣
 ↪linewidth=2,
                label=f'Mean estimate = ${estimates_random.mean():.2f}k')
axes[0].set_xlabel('Estimated Treatment Effect ($1000s)')
axes[0].set_ylabel('Frequency')
axes[0].set_title('Random Assignment: Unbiased Estimates')
axes[0].legend()
```

```python
axes[0].grid(True, alpha=0.3, axis='y')

# Right: Biased assignment
axes[1].hist(estimates_biased, bins=40, alpha=0.7, color='coral',␣
↪edgecolor='black')
axes[1].axvline(true_ate, color='red', linestyle='--', linewidth=2,␣
↪label=f'True ATE = ${true_ate:.2f}k')
axes[1].axvline(estimates_biased.mean(), color='green', linestyle='-',␣
↪linewidth=2,
                label=f'Mean estimate = ${estimates_biased.mean():.2f}k')
axes[1].set_xlabel('Estimated Treatment Effect ($1000s)')
axes[1].set_ylabel('Frequency')
axes[1].set_title('Selection Bias: Systematically Biased Estimates')
axes[1].legend()
axes[1].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

print("\nRandom Assignment:")
print(f"  Mean of estimates: ${estimates_random.mean():.2f}k")
print(f"  Bias: ${estimates_random.mean() - true_ate:.2f}k (close to 0  )")
print(f"  Standard error: ${estimates_random.std():.2f}k")

print("\nSelection Bias:")
print(f"  Mean of estimates: ${estimates_biased.mean():.2f}k")
print(f"  Bias: ${estimates_biased.mean() - true_ate:.2f}k (NOT close to 0  )")
print(f"  Standard error: ${estimates_biased.std():.2f}k")

print("\n  Key Insight: Randomization centers the distribution on the true ATE.
↪")
print("   Selection bias shifts the entire distribution, leading to systematic␣
↪error.")
```



```
Random Assignment:
  Mean of estimates: $5.03k
  Bias: $0.03k (close to 0  )
  Standard error: $1.03k

Selection Bias:
  Mean of estimates: $9.17k
  Bias: $4.17k (NOT close to 0  )
  Standard error: $0.84k

 Key Insight: Randomization centers the distribution on the true ATE.
   Selection bias shifts the entire distribution, leading to systematic error.
```

1.8 Part 5: Different Types of Average Treatment Effects

We've focused on the **Average Treatment Effect (ATE)**, but there are other quantities of interest:

1. **ATE**: E[Y(1) - Y(0)] - Average effect in the population
2. **ATT**: E[Y(1) - Y(0) | T=1] - Average effect for the treated
3. **ATU**: E[Y(1) - Y(0) | T=0] - Average effect for the untreated

These can differ when treatment effects are heterogeneous!

[16]:
```
# Create data where treatment effects vary with baseline characteristics
np.random.seed(123)
n = 1000

# Baseline ability (affects both Y(0) and treatment effect)
ability = np.random.normal(0, 1, n)

# Y(0) depends on ability
Y0 = 30 + 5 * ability + np.random.normal(0, 2, n)

# Treatment effect is LARGER for higher ability people
treatment_effect = 5 + 2 * ability + np.random.normal(0, 1, n)
Y1 = Y0 + treatment_effect

# Non-random treatment: higher ability people more likely to be treated
ability_norm = (ability - ability.mean()) / ability.std()
prob_T = 1 / (1 + np.exp(-0.5 * ability_norm))
T = np.random.binomial(1, prob_T)

# Calculate different ATEs
```

```python
ATE = treatment_effect.mean()
ATT = treatment_effect[T == 1].mean()
ATU = treatment_effect[T == 0].mean()

print("Heterogeneous Treatment Effects:\n")
print(f"ATE (Average Treatment Effect):          ${ATE:.2f}k")
print(f"ATT (Average Treatment Effect on Treated): ${ATT:.2f}k")
print(f"ATU (Average Treatment Effect on Untreated): ${ATU:.2f}k")
print(f"\nDifference (ATT - ATU):                  ${ATT - ATU:.2f}k")
print(f"\nInterpretation: Treated people benefit MORE from treatment than")
print(f"untreated people would (due to higher ability).")
```

```
Heterogeneous Treatment Effects:

ATE (Average Treatment Effect):          $4.93k
ATT (Average Treatment Effect on Treated): $5.40k
ATU (Average Treatment Effect on Untreated): $4.41k

Difference (ATT - ATU):                  $0.99k

Interpretation: Treated people benefit MORE from treatment than
untreated people would (due to higher ability).
```

1.9 Summary and Key Takeaways

1.9.1 Core Concepts

1. **Potential Outcomes Framework**:
   - Each unit has potential outcomes Y(1) and Y(0)
   - Causal effect = Y(1) - Y(0)
   - We can never observe both for the same unit (fundamental problem)
2. **Average Treatment Effect (ATE)**:
   - ATE = E[Y(1) - Y(0)]
   - What we want to estimate from data
   - Individual effects may vary (heterogeneity)
3. **Selection Bias**:
   - Naive comparison fails when treatment assignment is not random
   - E[Y|T=1] - E[Y|T=0] = ATE + Selection Bias
   - Selection Bias = E[Y(0)|T=1] - E[Y(0)|T=0]
4. **Randomization**:
   - Makes treatment independent of potential outcomes
   - Eliminates selection bias: E[Y(0)|T=1] = E[Y(0)|T=0]
   - Allows naive comparison to recover ATE
5. **Different Estimands**:
   - ATE: Average effect in population
   - ATT: Average effect for those who were treated
   - ATU: Average effect for those who were not treated

- These differ when effects are heterogeneous

1.9.2 Why This Matters

The potential outcomes framework gives us: - **Precise definitions** of what causal effects are - **Understanding** of why naive comparisons fail - **Justification** for randomized experiments - **Foundation** for more advanced methods (matching, IV, DiD, etc.)

In the next notebook, we'll learn about **Directed Acyclic Graphs (DAGs)**, another framework for thinking about causality that complements the potential outcomes approach.

1.10 Exercises

Try these to reinforce your understanding:

1. **Modify the simulation**: Change the selection mechanism in the biased assignment scenario. Make low Y(0) people more likely to get treatment instead of high Y(0). How does this affect the bias?
2. **Sample size**: Re-run the randomization simulation with n=20 instead of n=100. What happens to the distribution of estimates? Why?
3. **Heterogeneous effects**: Create a scenario where the treatment effect is negative for some people and positive for others. What is the ATE? What would happen if only people with positive effects selected into treatment?
4. **Three treatments**: Extend the framework to three treatments (T {0, 1, 2}). How many potential outcomes does each unit have? How many pairwise comparisons can you make?

1.11 Further Reading

- Rubin, D. B. (1974). "Estimating causal effects of treatments in randomized and nonrandomized studies." *Journal of Educational Psychology*, 66(5), 688-701.
- Holland, P. W. (1986). "Statistics and Causal Inference." *Journal of the American Statistical Association*, 81(396), 945-960.
- Imbens, G. W., & Rubin, D. B. (2015). *Causal Inference for Statistics, Social, and Biomedical Sciences*. Chapters 1-2.
- Hernán, M. A., & Robins, J. M. (2020). *Causal Inference: What If*. Chapters 1-2.

Next: DAGs Basics - Learn to represent and reason about causal relationships using graphs.